

AI Quiz and AI Chatbot: FRONTEND guide

For the Angular 16 app with the Midnight (dark navy and amber) theme. **All new files. Existing components are not rewritten.** Five tiny additions to existing files are listed in Part 4.

0. Status

- `ng build` succeeds with these files.
- Tested in a real headless browser against a pretend backend that follows the API contract: **46 checks, 0 failures** (chat open/close, Enter and Shift+Enter, history sent, course cards, 429 and 502 messages, quiz setup, answering, scoring, review, history list, "unanswered counts as wrong", AI-off note, admin and logged-out users blocked, phone width, logout hides the chatbot, no JavaScript errors).
- **Not tested:** against your real Spring backend and real Gemini. You need the backend guide done first.

1. What the user sees

- **Floating gold button** (bottom right) on every page, only for a logged-in CUSTOMER. It opens a chat panel with 3 suggestion buttons. Answers can show course cards that open the course details page.
- **AI Quiz** link in the navbar, page `/ai-quiz`: choose course, difficulty and number of questions, answer, submit, see marks, right answers and explanations, and your last quizzes.

2. Concepts

Module. The new `AiModule` holds both components. The chatbot is *exported* so `AppComponent` can place `<app-ai-chatbot>` on every page. The quiz has a route guarded by your existing `AuthGuard` (CUSTOMER only).

Why `AiModule` goes before `AppRoutingModule` in `imports`. Angular reads routes in import order. Your routing has a `**` catch-all. If `AiModule` came after it, `/ai-quiz` would hit the catch-all first.

Why the chatbot reads `auth.roles`. It appears when the role is CUSTOMER and clears itself on logout, with no page reload, using the same stream your navbar uses.

History. The chat is kept in the component only. Each message sends the last 8 turns so the server can give context. A full browser refresh starts a clean chat on purpose.

No answers in the browser. The quiz response has no correct answers. The page learns them only from the submit response.

3. Files (9 new)

#	File
1	models/ai.model.ts
2	services/ai-assistant.service.ts (your old ai.service.ts stays untouched)
3	modules/ai/ai.module.ts
4-6	components/ai-chatbot/ (ts, html, css)
7-9	components/ai-quiz/ (ts, html, css)

Two-person split: Person A: files 1 to 3 and 4 to 6 (chatbot) plus edits 4a, 4c. Person B: files 7 to 9 (quiz) plus edits 4b, 4d, 4e.

4. Five small additions to existing files

4a. constant.ts : inside the `API` object, after the `COURSE_AI_SEARCH` line:

```
// AI quiz and chatbot
AI_STATUS: '/api/ai/status',
AI_QUIZ_GENERATE: '/api/ai/quiz/generate',
AI_QUIZ_SUBMIT: '/api/ai/quiz/submit',
AI_QUIZ_HISTORY: '/api/ai/quiz/history',
AI_CHAT: '/api/ai/chat',
```

4b. app.module.ts : add the import line and put `AiModule` **before** `AppRoutingModule` :

```
import { AiModule } from './modules/ai/ai.module';
// ...
imports: [BrowserModule, BrowserAnimationsModule, HttpClientModule, SharedModule, AiModule,
AppRoutingModule],
```

4c. app.component.html : add one line at the end (after `<router-outlet>` and the footer if you have one):

```
<app-ai-chatbot></app-ai-chatbot>
```

4d. navbar.component.html : in the CUSTOMER block add after the "Available Courses" link, and the same in the mobile menu (`mat-menu-item` version):

```
<a routerLink="/ai-quiz" routerLinkActive="active">AI Quiz</a>
<!-- in the mat-menu: -->
<a mat-menu-item routerLink="/ai-quiz">AI Quiz</a>
```

4e. Nothing else. Styles use your existing Midnight variables (`--gold`, `--surface`, `--ink` ...), so `styles.css` is not touched.

5. Complete code

angularapp/src/app/models/ai.model.ts (new file)

```
/** Shapes of the AI quiz and AI chatbot API (see AiAssistantController in the backend). */

export type Difficulty = 'EASY' | 'MEDIUM' | 'HARD';
```

```

export interface QuizQuestionView {
  index: number;
  question: string;
  options: string[];
}

export interface QuizResponse {
  quizId: string;
  courseId: number;
  courseTitle: string;
  difficulty: Difficulty;
  questions: QuizQuestionView[];
}

export interface QuizResultItem {
  index: number;
  question: string;
  options: string[];
  yourAnswer: number | null;
  correctAnswer: number;
  correct: boolean;
  explanation: string;
}

export interface QuizResult {
  score: number;
  total: number;
  percent: number;
  items: QuizResultItem[];
}

export interface QuizHistoryItem {
  attemptId: number;
  courseId: number;
  courseTitle: string;
  difficulty: string;
  score: number;
  total: number;
  takenAt: string;
}

export interface ChatTurn {
  role: 'user' | 'bot';
  text: string;
}

export interface ChatCourse {
  courseId: number;
  courseType: string;
  coursePrice: number;
}

export interface ChatResponse {
  reply: string;
  courses: ChatCourse[];
  aiUsed: boolean;
}

```

angularapp/src/app/services/ai-assistant.service.ts (new file)

```

import { HttpClient } from '@angular/common/http';
import { Injectable } from '@angular/core';
import { Observable } from 'rxjs';
import { API, API_URL } from '../constant';
import { ChatResponse, ChatTurn, Difficulty, QuizHistoryItem, QuizResponse, QuizResult } from
'../models/ai.model';

/**

```

```

* AI quiz and AI chatbot calls. (The older AI course search stays in AiService.)
* The login token is added by AuthInterceptor, so nothing about tokens appears here.
*/
@Injectable({
  providedIn: 'root'
})
export class AiAssistantService {
  public apiUrl = API_URL;

  constructor(private http: HttpClient) {}

  status(): Observable<{ aiEnabled: boolean }> {
    return this.http.get<{ aiEnabled: boolean }>(`${this.apiUrl}${API.AI_STATUS}`);
  }

  generateQuiz(courseId: number, numQuestions: number, difficulty: Difficulty): Observable<QuizResponse> {
    return this.http.post<QuizResponse>(`${this.apiUrl}${API.AI_QUIZ_GENERATE}`, { courseId, numQuestions, difficulty });
  }

  submitQuiz(quizId: string, answers: (number | null)[]): Observable<QuizResult> {
    return this.http.post<QuizResult>(`${this.apiUrl}${API.AI_QUIZ_SUBMIT}`, { quizId, answers });
  }

  quizHistory(): Observable<QuizHistoryItem[]> {
    return this.http.get<QuizHistoryItem[]>(`${this.apiUrl}${API.AI_QUIZ_HISTORY}`);
  }

  chat(message: string, history: ChatTurn[]): Observable<ChatResponse> {
    return this.http.post<ChatResponse>(`${this.apiUrl}${API.AI_CHAT}`, { message, history });
  }
}

```

angularapp/src/app/modules/ai/ai.module.ts (new file)

```

import { NgModule } from '@angular/core';
import { RouterModule, Routes } from '@angular/router';
import { SharedModule } from '../../../shared/shared.module';
import { AuthGuard } from '../../../components/authguard/authguard.guard';
import { ROLES } from '../../../constant';
import { AiChatbotComponent } from '../../../components/ai-chatbot/ai-chatbot.component';
import { AiQuizComponent } from '../../../components/ai-quiz/ai-quiz.component';

const routes: Routes = [
  { path: 'ai-quiz', component: AiQuizComponent, canActivate: [AuthGuard], data: { roles: [ROLES.CUSTOMER] } }
];

/**
 * AI quiz page and the floating AI chatbot. The chatbot is exported so AppComponent can show it on every
 * page;
 * it only appears for a logged-in customer. This module must be imported in AppModule BEFORE
 * AppRoutingModule,
 * so its route is registered before the "*" fallback route.
 */
@NgModule({
  declarations: [AiChatbotComponent, AiQuizComponent],
  imports: [SharedModule, RouterModule.forChild(routes)],
  exports: [AiChatbotComponent]
})
export class AiModule {}

```

angularapp/src/app/components/ai-chatbot/ai-chatbot.component.ts (new file)

```

import { Component, ElementRef, HostListener, OnDestroy, OnInit, ViewChild } from '@angular/core';
import { Subscription } from 'rxjs';
import { AiAssistantService } from '../../../services/ai-assistant.service';

```

```

import { AuthService } from '../../services/auth.service';
import { ChatCourse, ChatTurn } from '../../models/ai.model';

interface Bubble {
  from: 'user' | 'bot';
  text: string;
  courses?: ChatCourse[];
}

/**
 * Floating AI assistant for customers. The conversation lives only in this component (no database):
 * every message sends the last few turns so the AI knows what was said before.
 */
@Component({
  selector: 'app-ai-chatbot',
  templateUrl: './ai-chatbot.component.html',
  styleUrls: ['./ai-chatbot.component.css']
})
export class AiChatbotComponent implements OnInit, OnDestroy {
  static readonly MAX_LENGTH = 500;
  static readonly HISTORY_TURNS = 8;

  role: string | null = null;
  open = false;
  draft = '';
  sending = false;
  error = '';
  bubbles: Bubble[] = [];
  readonly suggestions = ['Which course should I start with?', 'Find me a cheap beginner course', 'What have I enrolled in?'];
  readonly maxLength = AiChatbotComponent.MAX_LENGTH;

  @ViewChild('scroller') scroller?: ElementRef<HTMLElement>;
  @ViewChild('box') box?: ElementRef<HTMLTextAreaElement>;
  private sub?: Subscription;

  constructor(private ai: AiAssistantService, private auth: AuthService) {}

  ngOnInit(): void {
    this.sub = this.auth.role$.subscribe(r => {
      this.role = r;
      if (r !== 'CUSTOMER') { this.reset(); } // logged out or another user: forget the chat
    });
  }

  ngOnDestroy(): void {
    this.sub?.unsubscribe();
  }

  toggle(): void {
    this.open = !this.open;
    if (this.open) { setTimeout(() => this.box?.nativeElement.focus(), 0); this.scrollToBottom(); }
  }

  @HostListener('document:keydown.escape')
  onEscape(): void {
    if (this.open) { this.open = false; }
  }

  reset(): void {
    this.open = false;
    this.bubbles = [];
    this.draft = '';
    this.error = '';
    this.sending = false;
  }

  clear(): void {
    this.bubbles = [];
    this.error = '';
  }
}

```

```

useSuggestion(text: string): void {
  this.draft = text;
  this.send();
}

onEnter(ev: Event): void {
  if ((ev as KeyboardEvent).shiftKey) { return; }
  ev.preventDefault();
  this.send();
}

send(): void {
  const text = this.draft.trim();
  if (!text || this.sending || text.length > this.maxLength) { return; }
  const history: ChatTurn[] = this.bubbles
    .slice(-AiChatbotComponent.HISTORY_TURNS)
    .map(b => ({ role: b.from, text: b.text }));
  this.bubbles.push({ from: 'user', text });
  this.draft = '';
  this.error = '';
  this.sending = true;
  this.scrollDown();
  this.ai.chat(text, history).subscribe({
    next: res => {
      this.bubbles.push({ from: 'bot', text: res.reply, courses: res.courses });
      this.sending = false;
      this.scrollDown();
    },
    error: err => {
      this.sending = false;
      this.error = err.status === 429 || err.status === 400 || err.status === 503
        ? (err.error?.message || 'The assistant cannot answer right now.')
        : 'The assistant is unavailable. Please try again in a moment.';
      this.scrollDown();
    }
  });
}

trackByIndex(i: number): number {
  return i;
}

private scrollDown(): void {
  setTimeout(() => {
    const el = this.scroller?.nativeElement;
    if (el) { el.scrollTop = el.scrollHeight; }
  }, 0);
}
}

```

angularapp/src/app/components/ai-chatbot/ai-chatbot.component.html (new file)

```

<ng-container *ngIf="role === 'CUSTOMER'">
  <button type="button" class="fab" id="chatbotToggle" (click)="toggle()"
    [attr.aria-expanded]="open" aria-controls="chatbotPanel" [attr.aria-label]="open ? 'Close AI
assistant' : 'Open AI assistant'">
    <mat-icon>{{ open ? 'close' : 'auto_awesome' }}</mat-icon>
  </button>

  <section class="panel" id="chatbotPanel" *ngIf="open" role="dialog" aria-label="AI assistant">
    <header class="head">
      <div class="title">
        <mat-icon>auto_awesome</mat-icon>
        <div>
          <strong>AI assistant</strong>
          <span>Ask about courses and learning</span>
        </div>
      </div>
    </div>
  </section>
</ng-container>

```

```

        <button type="button" class="icon-btn" (click)="clear()" [disabled]="!bubbles.length" aria-
label="Clear chat" title="Clear chat">
        <mat-icon>delete_sweep</mat-icon>
    </button>
</header>

<div class="scroller" #scroller aria-live="polite">
    <div class="empty" *ngIf="!bubbles.length">
        <p>Hi! I can suggest courses, explain what they cover and help you plan what to learn next.</p>
        <button type="button" class="chip" *ngFor="let s of suggestions" (click)="useSuggestion(s)">{{ s }}
    </button>
    </div>

    <div class="row" *ngFor="let b of bubbles; trackBy: trackByIndex" [class.me]="b.from === 'user'">
        <div class="bubble">{{ b.text }}</div>
        <div class="cards" *ngIf="b.courses?.length">
            <a class="card" *ngFor="let c of b.courses" [routerLink]="['/course-details', c.courseId]"
(click)="open = false">
                <mat-icon>school</mat-icon>
                <span class="name">{{ c.courseType }}</span>
                <span class="price">{{ c.coursePrice | currency: 'INR' : 'symbol' : '1.0-0' }}</span>
            </a>
        </div>
    </div>

    <div class="row" *ngIf="sending"><div class="bubble typing" aria-label="The assistant is typing"><i>
</i><i><i><i><i></i></i></i></i></div></div>
        <div class="form-error" role="alert" *ngIf="error"><mat-icon>error_outline</mat-icon>{{ error }}
    </div>
</div>

    <form class="composer" (ngSubmit)="send()">
        <textarea #box id="chatbotInput" name="draft" rows="1" [(ngModel)]="draft" [maxLength]="maxLength"
placeholder="Type your question" aria-label="Your message"
(keydown.enter)="onEnter($event)"></textarea>
        <button type="submit" class="send" id="chatbotSend" [disabled]="sending || !draft.trim()" aria-
label="Send message">
            <mat-icon>send</mat-icon>
        </button>
    </form>
</section>
</ng-container>

```

angularapp/src/app/components/ai-chatbot/ai-chatbot.component.css (new file)

```

/* AI chatbot (Midnight): gold round button, dark panel */
.fab { position: fixed; right: 22px; bottom: 22px; z-index: 900; width: 56px; height: 56px; border: 0;
border-radius: 50%; cursor: pointer; display: grid; place-items: center; background: var(--gold); color:
#1a1205; box-shadow: var(--shadow-hover); transition: transform .15s ease; }
.fab:hover { transform: translateY(-2px); }
.fab mat-icon { font-size: 26px; width: 26px; height: 26px; }

.panel { position: fixed; right: 22px; bottom: 90px; z-index: 900; width: 380px; max-width: calc(100vw -
24px); height: 540px; max-height: calc(100vh - 120px); display: flex; flex-direction: column; background:
var(--surface); border: 1px solid var(--line); border-radius: var(--radius); box-shadow: var(--shadow);
overflow: hidden; }
.head { display: flex; align-items: center; justify-content: space-between; padding: 12px 14px; background:
var(--surface-2); border-bottom: 1px solid var(--line); }
.title { display: flex; align-items: center; gap: 10px; }
.title > mat-icon { display: grid; place-items: center; width: 34px; height: 34px; font-size: 20px; border-
radius: 10px; background: var(--gold); color: #1a1205; }
.title strong { display: block; font: 700 15px var(--display); color: var(--ink); }
.title span { font-size: 12.5px; color: var(--muted); }
.icon-btn { border: 0; background: transparent; color: var(--muted); cursor: pointer; border-radius: 8px;
padding: 6px; display: grid; place-items: center; }
.icon-btn:hover:not(:disabled) { color: var(--gold-text); background: var(--gold-soft); }
.icon-btn:disabled { opacity: .4; cursor: default; }

.scroller { flex: 1; overflow-y: auto; padding: 14px; display: flex; flex-direction: column; gap: 12px; }

```

```

.empty p { margin: 0 0 12px; color: var(--ink-2); font-size: 14.5px; line-height: 1.55; }
.chip { display: block; width: 100%; text-align: left; margin-bottom: 8px; padding: 9px 12px; border-
radius: 10px; border: 1px solid var(--line); background: var(--surface-2); color: var(--ink-2); font: 500
13.5px Inter, sans-serif; cursor: pointer; }
.chip:hover { border-color: var(--gold); color: var(--gold-text); }

.row { display: flex; flex-direction: column; align-items: flex-start; gap: 8px; }
.row.me { align-items: flex-end; }
.bubble { max-width: 88%; padding: 9px 13px; border-radius: 14px 14px 14px 4px; background: var(--surface-
2); color: var(--ink); font-size: 14.5px; line-height: 1.5; white-space: pre-wrap; overflow-wrap: anywhere;
border: 1px solid var(--line); }
.row.me .bubble { border-radius: 14px 14px 4px 14px; background: var(--gold); color: #1a1205; border-color:
transparent; }

.cards { display: flex; flex-direction: column; gap: 6px; width: 88%; }
.card { display: flex; align-items: center; gap: 8px; padding: 8px 10px; border-radius: 10px; border: 1px
solid var(--line); background: var(--surface); text-decoration: none; color: var(--ink); font-weight: 600;
font-size: 13.5px; }
.card:hover { border-color: var(--gold); }
.card mat-icon { color: var(--gold); font-size: 20px; width: 20px; height: 20px; }
.card .name { flex: 1; overflow: hidden; text-overflow: ellipsis; white-space: nowrap; }
.card .price { color: var(--gold-text); }

.typing { display: flex; gap: 5px; padding: 12px 14px; }
.typing i { width: 7px; height: 7px; border-radius: 50%; background: var(--muted); animation: blink 1.2s
infinite; }
.typing i:nth-child(2) { animation-delay: .2s; }
.typing i:nth-child(3) { animation-delay: .4s; }
@keyframes blink { 0%, 80%, 100% { opacity: .25; } 40% { opacity: 1; } }

.composer { display: flex; align-items: flex-end; gap: 8px; padding: 10px 12px; border-top: 1px solid var(-
-line); background: var(--surface-2); }
.composer textarea { flex: 1; resize: none; max-height: 96px; min-height: 40px; padding: 9px 12px; border-
radius: 10px; border: 1px solid var(--line); background: var(--bg); color: var(--ink); font: 400 14.5px
Inter, sans-serif; line-height: 1.4; }
.composer textarea::placeholder { color: var(--muted); }
.composer textarea:focus { outline: none; border-color: var(--gold); }
.send { width: 40px; height: 40px; border: 0; border-radius: 10px; cursor: pointer; display: grid; place-
items: center; background: var(--gold); color: #1a1205; }
.send:disabled { opacity: .4; cursor: default; }

@media (max-width: 520px) {
  .panel { right: 8px; bottom: 84px; width: calc(100vw - 16px); height: calc(100vh - 110px); }
  .fab { right: 14px; bottom: 14px; }
}
@media (prefers-reduced-motion: reduce) { .fab, .typing i { transition: none; animation: none; } }

```

angularapp/src/app/components/ai-quiz/ai-quiz.component.ts (new file)

```

import { Component, OnInit } from '@angular/core';
import { ActivatedRoute } from '@angular/router';
import { AiAssistantService } from '../../../services/ai-assistant.service';
import { CourseService } from '../../../services/course.service';
import { Course } from '../../../models/course.model';
import { Difficulty, QuizHistoryItem, QuizResponse, QuizResult } from '../../../models/ai.model';

type Step = 'setup' | 'loading' | 'taking' | 'submitting' | 'result';

/**
 * AI quiz page: pick a course, a difficulty and a number of questions, answer, then see the marks with
 * an explanation for every question. The correct answers only arrive after submitting (the server keeps
 them).
 */
@Component({
  selector: 'app-ai-quiz',
  templateUrl: './ai-quiz.component.html',
  styleUrls: ['./ai-quiz.component.css']
})
export class AiQuizComponent implements OnInit {

```

```

step: Step = 'setup';
courses: Course[] = [];
coursesLoaded = false;
aiEnabled = true;

courseId: number | null = null;
numQuestions = 5;
difficulty: Difficulty = 'MEDIUM';
readonly counts = [3, 5, 8, 10];
readonly levels: Difficulty[] = ['EASY', 'MEDIUM', 'HARD'];

quiz?: QuizResponse;
answers: (number | null)[] = [];
result?: QuizResult;
history: QuizHistoryItem[] = [];
error = '';

constructor(private ai: AiAssistantService, private courseService: CourseService, private route:
ActivatedRoute) {}

ngOnInit(): void {
  const preset = Number(this.route.snapshot.queryParamMap.get('courseId'));
  if (preset > 0) { this.courseId = preset; }
  this.courseService.viewAllCourses().subscribe({
    next: list => { this.courses = list; this.coursesLoaded = true; },
    error: () => { this.coursesLoaded = true; this.error = 'Could not load the courses. Please refresh
the page.'; }
  });
  this.ai.status().subscribe({ next: s => (this.aiEnabled = s.aiEnabled), error: () => {} });
  this.loadHistory();
}

get answered(): number {
  return this.answers.filter(a => a !== null).length;
}

get canStart(): boolean {
  return this.courseId !== null && this.step === 'setup' && this.aiEnabled;
}

get verdict(): string {
  const p = this.result?.percent ?? 0;
  return p >= 80 ? 'Great work!' : p >= 50 ? 'Good progress. A little more practice will help.' : 'Keep
practising. Review the explanations below.';
}

start(): void {
  if (this.courseId === null) { return; }
  this.error = '';
  this.step = 'loading';
  this.ai.generateQuiz(this.courseId, this.numQuestions, this.difficulty).subscribe({
    next: q => {
      this.quiz = q;
      this.answers = q.questions.map(() => null);
      this.step = 'taking';
      window.scrollTo({ top: 0 });
    },
    error: err => { this.step = 'setup'; this.error = this.message(err, 'The quiz could not be created.
Please try again.');
```

```

    next: r => { this.result = r; this.step = 'result'; this.loadHistory(); window.scrollTo({ top: 0 });
  },
  error: err => {
    // a quiz can be submitted once; if the server no longer knows it (expired), start again from the
  setup
    this.error = this.message(err, 'Your answers could not be submitted. Please try again. ');
    this.step = err.status === 404 ? 'setup' : 'taking';
  }
});
}

again(): void {
  this.step = 'setup';
  this.quiz = undefined;
  this.result = undefined;
  this.error = '';
}

letter(i: number): string {
  return String.fromCharCode(65 + i);
}

private loadHistory(): void {
  this.ai.quizHistory().subscribe({ next: h => (this.history = h), error: () => {} });
}

private message(err: any, fallback: string): string {
  return [400, 403, 404, 429, 503].includes(err?.status) ? (err.error?.message || fallback) : fallback;
}
}

```

angularapp/src/app/components/ai-quiz/ai-quiz.component.html (new file)

```

<div class="page narrow">
  <div class="page-head">
    <div>
      <h2>AI Quiz</h2>
      <p>Test yourself on any course. The AI writes fresh questions each time, and you get an explanation
for every answer.</p>
    </div>
  </div>

  <p class="form-note" *ngIf="!aiEnabled">The AI is not switched on for this platform yet, so quizzes
cannot be created.</p>
  <div class="form-error" role="alert" *ngIf="error"><mat-icon>error_outline</mat-icon>{{ error }}</div>

  <!-- 1. setup -->
  <section class="form-card setup" *ngIf="step === 'setup' || step === 'loading'">
    <mat-form-field appearance="outline" subscriptSizing="dynamic" class="full">
      <mat-label>Course</mat-label>
      <mat-select id="quizCourse" name="course" [(ngModel)]="courseId" [disabled]="step === 'loading'">
        <mat-option *ngFor="let c of courses" [value]="c.courseId">{{ c.courseType }}</mat-option>
      </mat-select>
    </mat-form-field>
    <p class="muted small" *ngIf="coursesLoaded && !courses.length">There are no courses yet.</p>

    <fieldset class="group">
      <legend>Difficulty</legend>
      <label class="pill" *ngFor="let l of levels" [class.on]="difficulty === l">
        <input type="radio" name="difficulty" [value]="l" [(ngModel)]="difficulty" [disabled]="step ===
'loading'" />{{ l | titlecase }}
      </label>
    </fieldset>

    <fieldset class="group">
      <legend>Number of questions</legend>
      <label class="pill" *ngFor="let n of counts" [class.on]="numQuestions === n">
        <input type="radio" name="count" [value]="n" [(ngModel)]="numQuestions" [disabled]="step ===
'loading'" />{{ n }}
      </label>
    </fieldset>
  </section>

```

```

</label>
</fieldset>

<button mat-flat-button color="primary" class="solid" id="startQuiz" type="button" (click)="start()"
[disabled]="!canStart">
  <mat-icon>auto_awesome</mat-icon>{{ step === 'loading' ? 'Writing your quiz...' : 'Start quiz' }}
</button>
</section>

<!-- 2. answering -->
<section *ngIf="(step === 'taking' || step === 'submitting') && quiz">
  <div class="bar">
    <strong>{{ quiz.courseTitle }}</strong>
    <span class="muted">{{ quiz.difficulty | titlecase }} &middot; {{ answered }} of {{
quiz.questions.length }} answered</span>
    <div class="track" aria-hidden="true"><div class="fill" [style.width.%]="answered * 100 /
quiz.questions.length"></div></div>
  </div>

  <article class="q form-card" *ngFor="let q of quiz.questions" [attr.id]="`question` + q.index">
    <h3><span class="num">{{ q.index + 1 }}</span>{{ q.question }}</h3>
    <div class="options" role="radiogroup" [attr.aria-label]="`Question ` + (q.index + 1)">
      <label class="opt" *ngFor="let o of q.options; let j = index" [class.on]="answers[q.index] === j">
        <input type="radio" [name]="`q` + q.index" [checked]="answers[q.index] === j"
(change)="pick(q.index, j)" [disabled]="step === 'submitting'" />
        <span class="letter">{{ letter(j) }}</span><span>{{ o }}</span>
      </label>
    </div>
  </article>

  <div class="actions">
    <span class="muted small" *ngIf="answered < quiz.questions.length">Unanswered questions count as
wrong.</span>
    <button mat-flat-button color="primary" class="solid" id="submitQuiz" type="button"
(click)="submit()" [disabled]="step === 'submitting'">
      {{ step === 'submitting' ? 'Marking...' : 'Submit answers' }}
    </button>
  </div>
</section>

<!-- 3. result -->
<section *ngIf="step === 'result' && result">
  <div class="score form-card" id="quizScore">
    <div class="big">{{ result.score }}<small>/{{ result.total }}</small></div>
    <div>
      <h3>{{ result.percent }}%</h3>
      <p>{{ verdict }}</p>
    </div>
  </div>

  <article class="q form-card" *ngFor="let it of result.items" [class.right]="it.correct"
[class.wrong]="!it.correct">
    <h3><span class="num">{{ it.index + 1 }}</span>{{ it.question }}</h3>
    <ul class="review">
      <li *ngFor="let o of it.options; let j = index"
[class.is-correct]="j === it.correctAnswer" [class.is-yours]="j === it.yourAnswer && j !==
it.correctAnswer">
        <span class="letter">{{ letter(j) }}</span><span class="txt">{{ o }}</span>
        <span class="tag" *ngIf="j === it.correctAnswer">Correct answer</span>
        <span class="tag bad" *ngIf="j === it.yourAnswer && j !== it.correctAnswer">Your answer</span>
      </li>
    </ul>
    <p class="muted small" *ngIf="it.yourAnswer === null">You did not answer this one.</p>
    <p class="why" *ngIf="it.explanation"><mat-icon>lightbulb</mat-icon>{{ it.explanation }}</p>
  </article>

  <div class="actions">
    <button mat-stroked-button class="ghost" type="button" (click)="again()">Choose another
course</button>
    <button mat-flat-button color="primary" class="solid" id="retryQuiz" type="button" (click)="again();
start()">Try new questions</button>

```

```

</div>
</section>

<!-- history -->
<section class="history" *ngIf="history.length && (step === 'setup' || step === 'result')">
  <h3>Your recent quizzes</h3>
  <ul>
    <li *ngFor="let h of history">
      <span class="h-title">{{ h.courseTitle }}</span>
      <span class="muted">{{ h.difficulty | titlecase }}</span>
      <strong>{{ h.score }}/{{ h.total }}</strong>
      <span class="muted">{{ h.takenAt | date: 'd MMM, h:mm a' }}</span>
    </li>
  </ul>
</section>
</div>

```

angularapp/src/app/components/ai-quiz/ai-quiz.component.css (new file)

```

/* AI quiz (Midnight) */
.small { font-size: 13px; }
.form-card { max-width: none; margin-left: 0; margin-right: 0; }
.full { width: 100%; }
.setup { display: flex; flex-direction: column; gap: 18px; }
.group { border: 0; padding: 0; margin: 0; display: flex; flex-wrap: wrap; gap: 8px; align-items: center; }
.group legend { width: 100%; padding: 0; margin-bottom: 8px; font: 700 13px var(--display); color: var(--gold-text); letter-spacing: .04em; }
.pill { position: relative; padding: 8px 18px; border-radius: 999px; border: 1px solid var(--line); background: var(--surface-2); color: var(--ink-2); font-weight: 600; font-size: 14px; cursor: pointer; }
.pill input { position: absolute; opacity: 0; inset: 0; cursor: pointer; }
.pill.on { background: var(--gold); border-color: var(--gold); color: #1a1205; }
.pill:focus-within { outline: 3px solid rgba(245,158,11,.55); outline-offset: 2px; }
#startQuiz { align-self: flex-start; }

.bar { margin-bottom: 16px; display: flex; flex-wrap: wrap; gap: 4px 14px; align-items: baseline; }
.bar strong { font: 700 17px var(--display); color: var(--ink); }
.track { flex-basis: 100%; height: 6px; border-radius: 6px; background: var(--surface-2); overflow: hidden; margin-top: 8px; }
.fill { height: 100%; background: var(--gold); transition: width .25s ease; }

.q { margin-bottom: 16px; }
.q h3 { display: flex; gap: 12px; align-items: flex-start; font-size: 17px; line-height: 1.45; margin-bottom: 14px; }
.num { flex: none; display: grid; place-items: center; width: 28px; height: 28px; border-radius: 8px; background: var(--gold-soft); color: var(--gold-text); font-size: 14px; }
.options { display: flex; flex-direction: column; gap: 8px; }
.opt { position: relative; display: flex; gap: 12px; align-items: center; padding: 11px 14px; border-radius: 10px; border: 1px solid var(--line); background: var(--surface-2); color: var(--ink-2); cursor: pointer; }
.opt:hover { border-color: rgba(245,158,11,.5); }
.opt input { position: absolute; opacity: 0; inset: 0; cursor: pointer; }
.opt.on { border-color: var(--gold); background: var(--gold-soft); color: var(--ink); }
.opt:focus-within { outline: 3px solid rgba(245,158,11,.55); outline-offset: 2px; }
.letter { flex: none; display: grid; place-items: center; width: 24px; height: 24px; border-radius: 50%; border: 1px solid var(--line); font-size: 12px; font-weight: 700; }
.opt.on .letter { background: var(--gold); border-color: var(--gold); color: #1a1205; }

.actions { display: flex; flex-wrap: wrap; gap: 12px; align-items: center; justify-content: flex-end; margin-top: 8px; }
.actions .muted { margin-right: auto; }

.score { display: flex; align-items: center; gap: 22px; margin-bottom: 16px; }
.big { font: 800 52px var(--display); color: var(--gold-text); line-height: 1; }
.big small { font-size: 24px; color: var(--muted); }
.score h3 { font-size: 22px; }
.score p { margin: 4px 0 0; color: var(--ink-2); }

.q.right { border-left: 4px solid var(--ok); }
.q.wrong { border-left: 4px solid var(--danger); }

```

```

.review { list-style: none; margin: 0; padding: 0; display: flex; flex-direction: column; gap: 6px; }
.review li { display: flex; gap: 12px; align-items: center; padding: 9px 12px; border-radius: 10px; border: 1px solid var(--line); color: var(--ink-2); }
.review li .txt { flex: 1; }
.review li.is-correct { border-color: var(--ok); background: rgba(52, 211, 153, .10); color: var(--ink); }
.review li.is-yours { border-color: var(--danger); background: rgba(248, 113, 113, .10); color: var(--ink); }
}
.tag { font-size: 12px; font-weight: 700; color: var(--ok); }
.tag.bad { color: var(--danger); }
.why { display: flex; gap: 8px; align-items: flex-start; margin: 14px 0 0; padding: 10px 12px; border-radius: 10px; background: var(--gold-soft); color: var(--ink); font-size: 14.5px; }
.why mat-icon { flex: none; color: var(--gold); font-size: 20px; width: 20px; height: 20px; }

.history { margin-top: 36px; }
.history h3 { margin-bottom: 12px; font-size: 18px; }
.history ul { list-style: none; margin: 0; padding: 0; display: flex; flex-direction: column; gap: 8px; }
.history li { display: grid; grid-template-columns: 1fr auto auto auto; gap: 16px; align-items: center; padding: 10px 14px; border-radius: 10px; background: var(--surface); border: 1px solid var(--line); font-size: 14px; }
.h-title { font-weight: 600; color: var(--ink); overflow: hidden; text-overflow: ellipsis; white-space: nowrap; }
@media (max-width: 600px) { .history li { grid-template-columns: 1fr auto; } .score { flex-direction: column; align-items: flex-start; } }
@media (prefers-reduced-motion: reduce) { .fill { transition: none; } }

```

6. Browser checks (after the backend works)

1. Logged out: no gold button. Open `/ai-quiz`: you land on login.
2. Log in as ADMIN: no gold button, `/ai-quiz` shows the error page.
3. Log in as CUSTOMER: gold button appears and "AI Quiz" is in the navbar.
4. Open the chat, click a suggestion, get an answer. Ask "tell me about java": course cards appear and open the details page.
5. Press Enter to send, Shift+Enter for a new line. Escape closes the panel.
6. Quiz: Start stays disabled until a course is chosen. Start, answer some, submit. Check the score, red and green marks and explanations.
7. Press F12, Network tab, open the `generate` response: it must contain **no** `correctIndex`.
8. Stop the backend key (no `GEMINI_API_KEY`): chat shows a plain answer, quiz page shows the "AI is not switched on" note.
9. Narrow the window to phone width: panel fits, no sideways scrolling.
10. Log out: the gold button disappears.

7. Troubleshooting

Problem	Cause
'app-ai-chatbot' is not a known element	<code>AiModule</code> not imported in <code>AppModule</code>
<code>/ai-quiz</code> shows the home or error page	<code>AiModule</code> placed after <code>AppRoutingModule</code>
401 on every AI call	token missing, log in again (the interceptor adds it)
Chat button never appears	you are not logged in as CUSTOMER
<code>API.AI_CHAT</code> does not exist	step 4a missed